

[BOJ] 1436(C++/cpp)

☒ 미완	☐
☐ 언어	C++
☐ 문제 출처	BOJ
☒ 진행 상황	성공
☐ 알고리즘 분류	브루트포스
☒ 복습 필요	☐
☐ 풀이날	@2025년 3월 8일
☐ 난도	실버 5
☒ 발전 가능성 여부	☐

문제

<https://www.acmicpc.net/problem/1436>

백준 1436번 - 영화감독 숀

N번째로 작은 '종말의 숫자'를 찾는 문제로, 종말의 숫자는 연속된 666이 포함된 숫자를 의미합니다.

풀이 1. 정수 연산을 이용한 탐색

아이디어

- 숫자를 1씩 증가시키면서, 각 자리수를 확인해 666이 연속으로 등장하는지 검사합니다.
- % 10을 이용해 숫자의 각 자리수를 확인하고, 666이 연속으로 등장하는 경우 카운트를 증가시킵니다.

코드

```
#include <iostream>
using namespace std;
```

```

int findNthApocalypseNumber(int n) {
    int num = 666;
    int count = 0;
    int result = 0;

    while (true) {
        int temp = num++;
        int consecutiveSixes = 0;

        while (temp != 0) {
            if (temp % 10 == 6) {
                consecutiveSixes++;
            } else {
                consecutiveSixes = 0;
            }

            temp /= 10;

            if (consecutiveSixes == 3) { // "666"이 포함됨
                count++;
                break;
            }
        }

        if (count == n) {
            return num - 1;
        }
    }
}

int main() {
    int n;
    cin >> n;

    int answer = findNthApocalypseNumber(n);
    cout << answer;
}

```

```
return 0;
}
```

시간 복잡도 분석

- $O(N * d)$, 여기서 d 는 각 숫자의 자리 수 (평균적으로 약 $O(N \log N)$ 에 가까움)
- 정수 연산만 사용하여 문자열 변환보다 성능이 좋을 것으로 생각됨

풀이 2. 문자열 변환을 이용한 탐색

아이디어

- 숫자를 문자열로 변환한 후, "666" 이 포함되는지 확인하는 방식
- `string::find()` 함수를 사용하여 "666" 이 포함된 경우 카운트를 증가

코드

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    int n;
    cin >> n;

    int count = 0;
    int num = 666;

    while (count < n) {
        string str = to_string(num++);

        if (str.find("666") != string::npos) {
            count++;
        }
    }
}
```

```
cout << num - 1;

return 0;
}
```

해결 과정에서 어려웠던 점

1. `to_string()` 변환이 빈번하게 일어나므로 성능이 좋을지 고민됨
2. `find()` 를 사용하면 간단하지만, 문자열 변환 과정에서 추가적인 연산이 필요

시간 복잡도 분석

- $O(N * d)$, 여기서 d 는 숫자의 평균 자릿수 (약 $O(N \log N)$)
- 정수 연산보다 약간 느릴 가능성이 있음

발전 가능한 아이디어

1. 메모이제이션 활용

- 666 을 포함하는 숫자들을 미리 저장해 두고 사용할 수도 있음 (예: 10000번째까지 미리 계산)

알게 된 내용

▼ `string::find()`

`find()` 함수는 문자열 내에서 특정 문자열이 처음 등장하는 위치를 반환합니다.

```
size_t find(const string& str) const;
```

- 만약 `str` 이 문자열 내에서 발견되면, 그 시작 인덱스(위치)를 반환합니다.
- 만약 `str` 이 문자열 내에 존재하지 않으면 `string::npos` 를 반환합니다.

`string::npos` 란?

`npos` 는 찾는 문자열이 존재하지 않을 때 반환되는 값으로, 사실상 최대 크기의 `size_t` 값 (보통 -1 로 표현됨)을 의미합니다.

```
string::npos == -1 // 대부분의 경우 참
```

`npos` 는 보통 `find()` 결과를 검사할 때 사용됩니다.

= string::npos 의 의미

```
if (str.find("666") != string::npos)
```

이 조건문의 의미는 **666** 이 문자열 안에 존재하는 경우를 확인하는 것입니다.

- `find()` 의 반환값이 `npos` 가 아니면 (`!= string::npos`): → 즉, `"666"` 이 문자열 안에 존재하면 참(**true**)
- `find()` 의 반환값이 `npos` 이면: → 즉, `"666"` 이 문자열 안에 없으면 거짓(**false**)

!= string::npos 를 사용하지 않는 경우

만약 아래처럼 `!= string::npos` 를 생략하고 `if (str.find("666"))` 만 사용한다면?

```
if (str.find("666"))
```

`find("666")` 가 0을 반환하는 경우(즉, 문자열의 가장 앞부분에 `"666"` 이 있는 경우), `if (0)` 이 되어 조건이 거짓(**false**)로 평가됩니다.

즉, `"666"` 이 문자열 앞부분에 있으면 정상적으로 감지되지 않는 문제가 발생합니다.

```
if (str.find("666") != string::npos)
```

이렇게 하면 `"666"`이 어디에 있는지 존재 여부만 확인하기 때문에 정확한 조건문이 됩니다.